

Equivalencia dependiente de contexto

Hazel Torres Nava¹, Leslie Paola Sánchez Victoria¹

¹ Facultad de Ciencias UNAM, Ciudad de México, México

e-mail: hazelt@ciencias.unam.mx, sanv.leslie@ciencias.unam.mx

Resumen—En la teoría de lenguajes de programación, las semánticas proporcionan mecanismos formales para describir el significado de los programas más allá de su estructura sintáctica. Dependiendo del enfoque —operacional, denotativo o axiomático— se modelan distintos aspectos del comportamiento computacional, tales como la ejecución paso a paso, la interpretación matemática o las propiedades verificables del programa. En este contexto surge naturalmente la noción de equivalencia de programas: determinar bajo qué criterios dos programas pueden considerarse semánticamente indistinguibles.

Palabras clave—equivalencia de programas, semántica axiomática, semántica denotativa, semántica operacional.

I. INTRODUCCIÓN

La equivalencia es un pilar fundamental de la teoría de la computación, ya que busca determinar si dos programas pueden considerarse iguales dentro de un mismo contexto.

Lo anterior implica demostrar que ambos programas son indistinguibles según un criterio formalmente definido. Sin embargo, el problema general de la equivalencia de programas es *indecidable* debido a su reducción al problema de la detención (*Halting Problem*). Por ello, no existe un algoritmo universal capaz de determinar si dos programas arbitrarios son equivalentes. En consecuencia, las distintas nociones de equivalencia deben centrarse en aspectos específicos del comportamiento de los programas.

Lo anterior conduce a reflexionar sobre cómo cambia la noción de equivalencia según el tipo de semántica utilizada. La respuesta radica en que la equivalencia de programas está determinada por una relación de observación dictada por el marco semántico elegido. Dependiendo de si se analiza el comportamiento observable paso a paso (semántica operacional), el objeto matemático abstracto que representa el programa (semántica denotativa) o las propiedades lógicas que pueden demostrarse sobre él (semántica axiomática); el criterio para considerar dos programas como *iguales* varía significativamente.

Comprender estas diferencias tiene un impacto directo y fundamental en áreas como la ingeniería de software y la verificación formal. Por ejemplo, la optimización de compiladores debe garantizar que las transformaciones de código —como la eliminación de código muerto, el plegamiento de constantes, entre otras— mejoren el rendimiento sin alterar la equivalencia semántica del programa original.

En este trabajo se estudian las principales nociones de equivalencia semántica derivadas de los enfoques operacional, denotativo y axiomático. Se analizan sus fundamentos teóricos, sus diferencias conceptuales y su utilidad para el razonamiento formal sobre programas.

II. EQUIVALENCIA DE PROGRAMAS

La noción de «equivalencia» determina si dos conceptos pueden considerarse iguales dentro de un contexto específico. Es una relación que se establece cuando dichos conceptos comparten características comunes en uno o varios aspectos relevantes.

En la teoría de lenguajes de programación, uno de los conceptos sobre los cuales puede definirse esta relación es el de programa. Un «programa» es la representación formal de un proceso dinámico, expresado mediante reglas sintácticas establecidas por un lenguaje de programación [1]; no se limita a ser una secuencia válida de caracteres, sino que incorpora un conjunto definido de comportamientos y resultados posibles durante su ejecución [2].

Estos comportamientos y resultados emergen de las construcciones fundamentales que conforman la mayoría de los programas: *expresiones* y *comandos*.

Las «expresiones» corresponden a construcciones que pueden evaluarse y reducirse a un valor, mientras que los «comandos» representan instrucciones asociadas a modificaciones sobre el estado del programa, generalmente sin producir un valor como resultado [3]. Muchos lenguajes de programación establecen una distinción explícita entre expresiones y comandos, lo que conduce a criterios de equivalencia semántica distintos según la naturaleza de la construcción analizada.

A partir de estas ideas surge el problema —generalmente indecible— de la *equivalencia de programas*: determinar si dos programas son equivalentes bajo la misma definición de equivalencia [4].

II-A. Equivalencia en contexto estático

La «equivalencia de tipos» es la relación definida por un sistema de tipos para determinar cuándo dos expresiones de tipo representan el mismo tipo [5]. Esta equivalencia puede clasificarse en dos formas [6]:

- **Equivalencia estructural.** Dos tipos son equivalentes si poseen la misma estructura; es decir, si fueron construidos utilizando los mismos constructores de tipos y sus componentes internos tienen los mismos tipos organizados de la misma manera.
- **Equivalencia por nombre.** Dos tipos son equivalentes solo si provienen de la misma declaración de tipo.

II-B. Equivalencia en contexto dinámico

La «equivalencia observacional» o «equivalencia contextual» establece que dos programas son equivalentes si cualquier ocurrencia del primero en un programa completo puede

ser reemplazada por el segundo sin afectar los resultados observables de su ejecución [7]; es decir, no existe ningún contexto del programa completo que permita distinguir aquel intercambio entre programas.

En otras palabras, dos programas son equivalentes si, aun teniendo implementaciones internas distintas, exhiben la misma funcionalidad al producir los mismos comportamientos observables ante las mismas entradas.

Lo anterior requiere definir la equivalencia de manera específica para expresiones y comandos: dos expresiones son equivalentes si se evalúan al mismo resultado en todo estado, mientras que dos comandos son equivalentes si, para cualquier estado inicial, ambos divergen o ambos terminan en el mismo estado final [8].

III. DISTINTAS PERSPECTIVAS

Para determinar si dos programas son contextualmente equivalentes, no basta con comparar sus valores finales –en caso de existir– sino que también deben considerarse sus efectos sobre el estado del programa y cualquier resultado o consecuencia en la ejecución que permita distinguir un programa de otro.

Lo anterior requiere describir el comportamiento del programa en términos de cómo sus instrucciones modifican el estado del sistema durante la ejecución, aspecto abordado por la semántica dinámica del lenguaje [9].

Entre los principales enfoques para describir el comportamiento dinámico se encuentran la semántica operacional, la semántica denotativa y la semántica axiomática.

III-A. Semántica operacional

Este enfoque semántico describe paso a paso cómo se ejecuta un programa, considerando los estados intermedios además del resultado final [10].

La semántica operacional se formaliza mediante un sistema de transiciones que define dos tipos de configuraciones:

- $\langle S, s \rangle$ indica que la instrucción S será ejecutada desde el estado s , y
- s representa un estado final

Existen dos enfoques para especificar la semántica operacional, los cuales difieren en la manera de definir la relación de transición [10].

La relación de transición se define mediante reglas de inferencia, cuya aplicación sucesiva permite construir árboles de derivación que representan formalmente las distintas etapas de ejecución de un programa. Esto permite razonar formalmente sobre los programas y sus propiedades, ya que dichos árboles justifican los pasos de ejecución necesarios para demostrar una determinada propiedad [10].

III-A1. Semántica natural: También conocida como *semántica de paso grande*, es un modelo de la semántica operacional que describe cómo se obtiene directamente el resultado final de la evaluación de las expresiones.

La «semántica natural» de un lenguaje T está dada por [11]:

- un dominio V de valores semánticos, y

- una relación de transición entre términos de T y valores en V , que asocia a cada término los valores que pueden obtenerse mediante su ejecución completa.

Esta relación describe cómo la ejecución de una instrucción transforma un estado inicial en un estado final [10].

Una transición en semántica de paso grande se escribe

$$\langle S, s \rangle \Rightarrow s'$$

para representar que la evaluación de S en el estado s da lugar al estado s' .

Si el estado s' existe, se dice que la ejecución de S *termina*; en caso contrario, se dice que la ejecución *diverge*.

Definida la relación de transición para la semántica de paso grande, se puede definir formalmente cuándo dos programas se consideran equivalentes.

Definición 1 (Equivalencia en semántica natural). Dos programas S_1 y S_2 son semánticamente equivalentes, si para todos los estados s y s'

$$\langle S_1, s \rangle \Rightarrow s' \quad \text{si y solo si} \quad \langle S_2, s \rangle \Rightarrow s'$$

La definición anterior establece que dos programas son equivalentes si, partiendo del mismo estado inicial, terminan exactamente en los mismos estados finales.

Los estados finales dependen de la categoría sintáctica considerada. En el caso de las expresiones, la equivalencia se define por la obtención del mismo valor de evaluación. En el caso de los comandos, la equivalencia se caracteriza por producir el mismo estado final cuando se ejecutan desde un mismo estado inicial.

Además, esta definición considera equivalentes a dos programas cuando, para un mismo estado inicial, ninguno de ellos puede derivar un valor o estado final. En consecuencia, la definición no distingue entre programas que se atascan y programas que divergen, ya que en ambos casos no existe una derivación hacia un resultado final.

III-A2. Semántica estructural: También conocida como *semántica de paso pequeño*, su propósito es describir cómo se llevan a cabo los pasos individuales de las ejecuciones [10].

La «semántica estructural» de un lenguaje T está dada por [11]:

- un dominio S de estados de ejecución, y
- una relación de transición en S que relaciona cada estado con todos los estados a los que se puede llegar realizando un solo paso de ejecución.

La relación de transición tiene la forma

$$\langle S, s \rangle \rightarrow \gamma$$

y describe un paso individual en la ejecución de S .

Si γ es de la forma:

- $\langle S', s' \rangle$ la ejecución no ha terminado y la configuración resultante representa el cálculo restante.
- s' la ejecución de S desde el estado s ha finalizado en el estado s' .

Una configuración $\langle S, s \rangle$ se considera *atascada* si no existe una configuración γ tal que $\langle S, s \rangle \rightarrow \gamma$.

Dado un programa S y un estado s , una secuencia de derivación desde la configuración inicial $\langle S, s \rangle$ puede ser:

- *finita*

$$\gamma_0, \gamma_1, \dots, \gamma_k$$

tal que $\gamma_0 = \langle S, s \rangle$ y $\gamma_i \rightarrow \gamma_{i+1}$ para todo $0 \leq i < k$, donde γ_k es una configuración terminal o atascada.

- *infinita*

$$\gamma_0, \gamma_1, \gamma_2, \dots$$

tal que $\gamma_0 = \langle S, s \rangle$ y $\gamma_i \rightarrow \gamma_{i+1}$ para todo $i \geq 0$.

La ejecución de un programa S desde un estado s se considera que *termina* si existe una secuencia de derivación finita desde $\langle S, s \rangle$ hasta una configuración terminal. En tal caso se denota $\langle S, s \rangle \rightarrow^* \gamma$.

Por otro lado, se considera que la ejecución *diverge* si existe una secuencia de derivación infinita desde $\langle S, s \rangle$.

Una vez definidas las nociones asociadas a la relación de transición, se presenta la definición formal de equivalencia en semántica de paso pequeño.

Definición 2 (Equivalencia en semántica estructural). Dos programas S_1 y S_2 son semánticamente equivalentes, si para todos los estados s

- $\langle S_1, s \rangle \rightarrow^* \gamma$ si y solo si $\langle S_2, s \rangle \rightarrow^* \gamma$ donde γ es una configuración atascada o terminal, y
- existe una derivación infinita iniciada en $\langle S_1, s \rangle$ si y solo si existe una derivación infinita iniciada en $\langle S_2, s \rangle$.

A grandes rasgos, la equivalencia en semántica estructural es análoga a la equivalencia en semántica natural. La principal diferencia radica en que la semántica de paso pequeño describe explícitamente cada transición de la ejecución, proporcionando una caracterización más detallada del comportamiento de los programas.

Dos programas son equivalentes si, para todo estado inicial, alcanzan exactamente las mismas configuraciones terminales o atascadas mediante un número finito de transiciones. En particular, cuando ambos programas se atascan, la equivalencia se mantiene si ninguno de ellos puede continuar su ejecución mediante la aplicación de alguna regla de transición.

Asimismo, la definición contempla programas que no terminan. En este caso, dos programas son equivalentes cuando, para todo estado inicial, ninguno de ellos alcanza un valor o estado final debido a que sus ejecuciones pueden prolongarse indefinidamente.

III-B. Semántica denotativa

Este tipo de semántica se interesa por describir qué efecto produce la ejecución de un programa [10], abstrayendo los detalles de su evaluación e implementación mediante su representación en términos matemáticos. En lugar de explicar cómo se ejecuta un programa, se enfoca en caracterizar qué significa.

La «semántica denotativa» de un lenguaje T está dada por [11]:

- un dominio V de valores semánticos, y

- una función semántica

$$\llbracket \cdot \rrbracket : T \rightarrow V,$$

que asigna a cada término $t \in T$ un valor semántico $\llbracket t \rrbracket \in V$.

La notación $\llbracket \cdot \rrbracket$, conocida como *corchetes semánticos*, denota el resultado de interpretar un término dentro de un modelo matemático del lenguaje [11].

La idea es asignar a cada categoría sintáctica una función semántica que asocie cada construcción del lenguaje con un objeto matemático —frecuentemente una función— que describa su comportamiento. Este enfoque se basa en el principio de composicionalidad: el significado de una construcción compuesta se define únicamente en función del significado de sus subcomponentes [10].

La composicionalidad garantiza que términos denotacionalmente iguales pueden sustituirse dentro de cualquier construcción sin alterar su significado, lo que justifica el uso del razonamiento ecuacional [11]. Asimismo, dado que la semántica denotacional se define por inducción estructural sobre la sintaxis, en la práctica es natural emplear inducción estructural para demostrar propiedades.

A continuación, se presenta la equivalencia entre programas mediante una modelación matemática.

Definición 3 (Equivalencia en semántica denotativa). Dos programas S_1 y S_2 son semánticamente equivalentes si y solo si

$$\llbracket S_1 \rrbracket = \llbracket S_2 \rrbracket$$

Esta definición de equivalencia reduce el problema a la igualdad matemática de los valores denotados. Es decir, dos programas son equivalentes cuando sus respectivas funciones semánticas lo son, lo que implica demostrar la *extensionalidad* de las mismas.

Por lo tanto, aunque los programas posean estructuras sintácticas distintas o sigan estrategias de ejecución diferentes, ambos son indistinguibles desde el punto de vista denotacional si representan exactamente el mismo comportamiento abstracto.

III-C. Semántica axiomática

El enfoque axiomático define el significado de los programas en términos de especificaciones lógicas sobre su comportamiento, centrándose en las propiedades que pueden demostrarse sobre los estados del programa antes y después de su ejecución [12]. La idea central es expresar afirmaciones lógicas sobre dicha ejecución.

Una «afirmación» es una *terna de Hoare* expresada de la forma

$$\{P\} S \{Q\}$$

donde

S denota un programa y,
 P, Q son predicados lógicos.

Esta terna de *correctitud parcial* expresa que toda ejecución terminante de S que parte de un estado donde se cumple

la precondition P , concluye en un estado que satisface la postcondición Q .

La «semántica axiomática» de un lenguaje T está dada por [13]:

- un conjunto de axiomas sobre T
- un conjunto de reglas de inferencia que permiten establecer la veracidad de las afirmaciones.

Este tipo de semántica puede analizarse desde dos perspectivas fundamentales de la lógica matemática: la «teoría de modelos» y la «teoría de la demostración» [8].

III-C1. En la teoría de modelos: La teoría de modelos estudia las relaciones entre los lenguajes formales y las estructuras matemáticas, siendo la «verdad» el puente que conecta ambos ámbitos [14]. Desde esta perspectiva, la semántica axiomática se fundamenta en un concepto formal de *validez* para las afirmaciones sobre programas.

Se dice que una terna de Hoare es «semánticamente válida» si y solo si, para todo estado s , se cumple que [10]

$$(s \models P \wedge \langle S, s \rangle \Rightarrow s') \implies s' \models Q.$$

En tal caso, se escribe

$$\models \{P\} S \{Q\}.$$

Es decir, una tripleta es *válida* cuando, para cualquier estado inicial s que satisfaga la precondition P , toda ejecución terminante del programa S conduce a un estado s' que satisface la postcondición Q [15].

Esta noción de validez se apoya en un modelo semántico basado en transiciones de estados, que describe formalmente el comportamiento de los programas [8]. En consecuencia, el significado de una terna de Hoare depende de cómo las ejecuciones del programa, descritas mediante transiciones de estado, satisfacen la especificación dada.

A partir de esta caracterización semántica puede definirse la equivalencia entre programas.

Definición 4 (Equivalencia en semántica axiomática y teoría de modelos). Dos programas S_1 y S_2 son semánticamente equivalentes si, para toda precondition P y toda postcondición Q ,

$$\models \{P\} S_1 \{Q\} \quad \text{si y solo si} \quad \models \{P\} S_2 \{Q\}.$$

Desde la teoría de modelos, dos programas comparten el mismo significado si son indistinguibles ante cualquier especificación lógica; es decir, cualquier descripción válida sobre el primer programa debe ser mutuamente válida para el segundo bajo el modelo de estados subyacente.

III-C2. En la teoría de la demostración: La teoría de la demostración, o teoría de la prueba, estudia las demostraciones en los sistemas lógicos, analizando su estructura, los procedimientos para transformarlas y las traducciones sintácticas entre teorías formales [16].

En este sentido, la semántica axiomática se centra en la derivabilidad de las afirmaciones sobre programas. En particular, una terna de Hoare se considera válida cuando puede obtenerse a partir de los axiomas y reglas de inferencia del sistema lógico. Esto se expresa mediante la notación

$$\vdash \{P\} S \{Q\}$$

que indica que la terna de Hoare puede demostrarse sintácticamente.

La equivalencia axiomática de programas, en la teoría de la prueba, se define de la siguiente manera.

Definición 5 (Equivalencia en semántica axiomática y teoría de la demostración). Dos programas S_1 y S_2 son derivacionalmente equivalentes si, para toda precondition P y toda postcondición Q ,

$$\vdash \{P\} S_1 \{Q\} \quad \text{si y solo si} \quad \vdash \{P\} S_2 \{Q\}.$$

A diferencia del enfoque basado en teoría de modelos, donde el significado de una terna se define mediante las ejecuciones del programa sobre estados, la teoría de la demostración interpreta las afirmaciones en términos de su derivación sintáctica. Es decir, el significado de una especificación está determinado por las reglas que permiten construir una demostración de ella [8]. Por ello, dos programas son equivalentes cuando satisfacen las mismas especificaciones que pueden derivarse a partir de las reglas de inferencia.

IV. EJEMPLO

Para introducir la noción de equivalencia de programas, se desarrolla la semántica del lenguaje de programación imperativo miniatura COMM [17]. Este lenguaje contiene construcciones básicas como asignaciones, composición secuencial, condicionales y ciclos *while*.

La sintaxis abstracta de COMM se define inductivamente como sigue.

```

A ::= n | x | A + A | A - A | A * A
B ::= true | false | B and B | not B | A < A | A = A
C ::= skip | new x := A in C | print A | x := A | C; C
    | if B then C else C end | while B do C end

```

Figura 1. Sintaxis abstracta del lenguaje COMM.

El objetivo es demostrar la siguiente equivalencia entre dos programas de COMM.

Proposición 1. Sea x un identificador y sea a una expresión aritmética. Entonces,

$$\text{new } x := a \text{ in skip} \equiv \text{skip}$$

Aunque la equivalencia anterior intuitivamente expresa que la creación de una variable local seguida de una instrucción sin efecto observable no altera el comportamiento del programa, cada enfoque semántico la formaliza de manera diferente.

Por ello, se propone demostrar dicha proposición desde las perspectivas operacional, denotativa y axiomática, con el objetivo de analizar y contrastar los criterios que cada una utiliza para establecer la equivalencia entre programas.

Adicionalmente, las demostraciones se desarrollan con el apoyo del asistente de pruebas ROCQ, que funciona como herramienta para la formalización y verificación de dichas semánticas.

La implementación en ROCQ de la semántica operacional y axiomática de COMM se basa en el trabajo de [8].

IV-A. Semántica operacional de COMM

La Proposición 1 puede formalizarse mediante la semántica operacional, en particular mediante la semántica de paso grande, de la siguiente manera.

Lema 1. Para todo identificador x , toda expresión aritmética a y todo estado s ,

$$\begin{aligned} \langle \text{new } x := a \text{ in skip}, s \rangle &\Rightarrow s' \\ \iff \\ \langle \text{skip}, s \rangle &\Rightarrow s' \end{aligned}$$

En ROCQ, la equivalencia de programas bajo semántica natural se formaliza mediante la siguiente definición. Dos programas son equivalentes si inducen exactamente la misma relación entre estados iniciales y finales.

Código 1. Definición de equivalencia operacional en Rocq

```
Definition equiv_operacional (c1 c2 : com) : Prop :=
forall st st',
  st =[ c1 ] => st' <-> st =[ c2 ] => st'.
```

La formalización del Lema 1 y su correspondiente demostración en ROCQ se muestran a continuación.

Código 2. Programas equivalentes con semántica natural en Rocq

```
Example equiv_local_skip :
forall x a,
  equiv_operacional
    <{ new x := a in skip }>
    <{ skip }>.
```

Proof.

```
split .
- intros .
  inversion H. subst .
  inversion H7. subst .
  rewrite restore_update .
  apply E_Skip.
- intros .
  inversion H. subst .
  rewrite <- (restore_update st' x (aeval st' a)).
  apply E_New with
    (n := aeval st' a)
    (o := st' x)
    (st' := (x !-> aeval st' a ; st')).
  + rewrite restore_update . reflexivity .
  + rewrite restore_update . reflexivity .
  + rewrite restore_update . apply E_Skip.
```

Qed.

La demostración comienza descomponiendo la doble implicación en dos direcciones. En cada una de ellas se aplica inversión sobre las derivaciones de evaluación, lo que permite identificar los constructores de la sintaxis abstracta y las reglas semánticas utilizadas para obtenerlas. Dichos constructores reflejan directamente las reglas de inferencia empleadas en la construcción del árbol de derivación correspondiente.

Cabe destacar que, para completar la demostración, fue necesario probar un lema auxiliar sobre estados. Dicho lema establece que actualizar una variable con un valor arbitrario y posteriormente restaurar su valor original permite recuperar exactamente el estado previo.

Esta demostración ilustra cómo la semántica operacional resulta especialmente adecuada para razonar sobre estados mutables, ya que permite seguir de manera precisa los cambios producidos en el estado por cada construcción del lenguaje.

IV-B. Semántica denotativa de COMM

En términos de semántica denotativa, la Proposición 1 se expresa formalmente como sigue.

Lema 2. Para todo identificador x y toda expresión aritmética a ,

$$\llbracket \text{new } x := a \text{ in skip} \rrbracket = \llbracket \text{skip} \rrbracket$$

En ROCQ, la equivalencia denotacional se formaliza mediante la siguiente definición.

Dos comandos son equivalentes si, para todo estado, sus funciones semánticas producen el mismo estado resultante.

Código 3. Definición de equivalencia denotativa en Rocq

```
Definition equiv_denotativa (c1 c2 : com) : Prop :=
forall st,
  [[ c1 ]] st = [[ c2 ]] st.
```

La formalización del Lema 2 y su correspondiente demostración en ROCQ se muestran a continuación.

Código 4. Programas equivalentes con semántica denotativa en Rocq

```
Example equiv_local_skip :
forall x a,
  equiv_denotativa
    <{ new x := a in skip }>
    <{ skip }>.
```

Proof.

```
intros x a st.
simpl .
unfold t_update .
apply functional_extensionality .
intro x'.
destruct (String.eqb x x') eqn:H.
- apply String.eqb_eq in H.
  rewrite H.
  reflexivity .
- reflexivity .
```

Qed.

La semántica denotativa permite capturar directamente el efecto global de un programa sin describir explícitamente cada paso de su ejecución. Como consecuencia, la demostración resulta considerablemente más sencilla que en el caso operacional.

La definición de la función semántica traduce el objetivo de la prueba en una igualdad entre funciones de estado. Tras expandir las definiciones sintácticas y del entorno local, la prueba se apoya en el principio de extensionalidad funcional para evaluar el comportamiento de los estados ante cualquier identificador arbitrario. Esto permite que el resto de la demostración se resuelva mediante simplificación algebraica y análisis de casos, evitando por completo el manejo de relaciones inductivas.

A diferencia de la semántica operacional, donde fue necesario analizar las reglas de evaluación y reconstruir derivaciones, en la semántica denotativa basta demostrar que ambos programas inducen la misma transformación sobre los estados.

IV-C. Semántica axiomática de COMM

La Proposición 1, formulada en términos de equivalencia axiomática mediante la teoría de modelos, es la siguiente.

Lema 3. Para todo identificador x y expresión aritmética a ,

$$\begin{aligned} \models \{P\} \text{ new } x := a \text{ in skip } \{Q\} \\ \iff \\ \models \{P\} \text{ skip } \{Q\} \end{aligned}$$

La equivalencia axiomática en ROCQ se define como la equivalencia entre ternas de Hoare válidas bajo cualquier par de aserciones.

Código 5. Definición de equivalencia axiomática en Rocq

```
Definition equiv_axiomatica (c1 c2 : com) : Prop :=
forall P Q,
  {{P}} c1 {{Q}} <-> {{P}} c2 {{Q}}.
```

Dado que la validez de las ternas se fundamenta en un modelo de transiciones, el siguiente teorema demuestra que la equivalencia en semántica operacional, establecida por `cequiv`, es un criterio suficiente para garantizar la equivalencia axiomática.

Código 6. Equivalencia bajo el modelo de estados para la semántica axiomática en Rocq

```
Lemma cequiv_valid_hoare :
forall c1 c2,
  cequiv c1 c2 -> equiv_axiomatica c1 c2.
```

Proof.

```
intros c1 c2 Heq P Q.
split .
- intros H st st' Hc HP.
  apply H with (st := st) (st' := st').
  + apply Heq.
  + exact Hc.
  + exact HP.
- intros H st st' Hc HP.
  apply H with (st := st) (st' := st').
  + apply Heq.
  + exact Hc.
  + exact HP.
```

Qed.

Con este teorema, demostrar la equivalencia de dos programas bajo el enfoque axiomático se reduce a demostrar su equivalencia en el modelo de estados subyacente.

Código 7. Programas equivalentes con semántica axiomática en Rocq

```
Example equiv_local_skip :
forall x a,
  equiv_axiomatica
    <{ new x:=a in skip }>
    <{ skip }>.
```

Proof.

```
intros .
apply cequiv_valid_hoare .
split .
- intros .
  inversion H. subst .
  inversion H7. subst .
  rewrite restore_update .
  apply E_Skip.
- intros .
  inversion H. subst .
  rewrite <- (restore_update st' x (aeval st' a)).
  apply E_New with
    (n := aeval st' a)
    (o := st' x)
    (st' := (x !-> aeval st' a ; st')).
  + rewrite restore_update . reflexivity .
  + rewrite restore_update . reflexivity .
  + rewrite restore_update . apply E_Skip.
```

Qed.

Se observa que la principal diferencia entre la demostración anterior respecto a la versión operacional se centra en la aplicación del teorema `cequiv_valid_hoare`.

La semántica axiomática razona sobre el comportamiento del programa y las garantías que debe cumplir durante la ejecución pero demostrar equivalencia requiere razonar sobre las transiciones explícitas del modelo de estados.

V. CONCLUSIÓN & DISCUSIÓN

Como el problema general de determinar si dos programas son equivalentes es indecidible, no existe un procedimiento universal capaz de resolver esta cuestión para cualquier par de programas. En consecuencia, el estudio de la equivalencia debe abordarse mediante criterios de indistinguibilidad establecidos por aspectos particulares del comportamiento computacional.

Entre los enfoques más relevantes se encuentran la *equivalencia operacional*, la *equivalencia denotativa* y la *equivalencia axiomática*, cada una asociada a una forma distinta de describir y razonar sobre el significado de los programas.

A través de la formalización de estas tres nociones de equivalencia para el lenguaje COMM mediante el asistente de pruebas ROCQ, este trabajo permitió constatar que una misma propiedad puede requerir estrategias de demostración significativamente distintas según el marco semántico adoptado. Mientras que las demostraciones operacionales exigen reconstruir derivaciones y analizar explícitamente las transformaciones de estado producidas durante la ejecución, las demostraciones denotativas se apoyan en propiedades matemáticas de las funciones que representan el significado de los programas, como la extensionalidad funcional. Por su parte, la equivalencia axiomática basada en teoría de modelos mostró cómo el razonamiento sobre propiedades lógicas de los programas puede fundamentarse en la estructura del modelo de estados subyacente.

En conjunto, los resultados obtenidos muestran que las distintas nociones de equivalencia no deben entenderse como conceptos rivales, sino como perspectivas complementarias para estudiar el comportamiento de los programas. De manera general, dos programas pueden considerarse equivalentes cuando resultan indistinguibles respecto de los criterios observables establecidos por una determinada semántica: en la semántica operacional, la equivalencia se establece a partir del comportamiento de ejecución y de las transformaciones de estado producidas por los programas; en la semántica denotativa, se determina mediante la igualdad de las funciones que representan su significado matemático; y en la semántica axiomática, se fundamenta en la imposibilidad de distinguir los programas mediante las propiedades lógicas que satisfacen.

Comprender sus diferencias, alcances y relaciones proporciona una base teórica sólida para áreas como la verificación formal, el diseño de lenguajes de programación y la optimización segura de compiladores. En estos contextos, la capacidad de demostrar rigurosamente que una transformación preserva el significado de un programa constituye un requisito fundamental para garantizar la corrección y confiabilidad de los sistemas de software.

V-A. Trabajo a futuro

Si bien este trabajo permitió formalizar y comparar distintas nociones de equivalencia de programas para el lenguaje COMM, permanecen abiertos diversos problemas cuya exploración podría ampliar tanto el alcance teórico como el grado de formalización de los resultados obtenidos.

En primer lugar, el estudio se centró exclusivamente en propiedades relacionadas con la semántica dinámica de los programas. No obstante, en la Sección II se introdujo la noción de equivalencia de tipos, la cual pertenece al ámbito de la semántica estática. Una posible extensión consiste en analizar las distintas formas de equivalencia asociadas a los sistemas de tipos y estudiar las relaciones existentes entre las nociones de equivalencia estática y dinámica dentro de un mismo lenguaje de programación.

Por otra parte, la formalización de la semántica denotativa de COMM se restringió a programas sin constructores iterativos. En consecuencia, resulta de interés extender la implementación en ROCQ para incorporar la instrucción *while*. Esta extensión requiere el estudio de herramientas matemáticas más avanzadas, particularmente la teoría de dominios y la teoría de puntos fijos, las cuales permiten modelar formalmente el significado de las construcciones recursivas e iterativas.

Finalmente, el análisis de la semántica axiomática reveló la existencia de dos nociones de equivalencia: una basada en la teoría de modelos y otra basada en la teoría de la prueba. En este trabajo únicamente se desarrolló la primera de ellas. Una línea natural de investigación consiste en formalizar la equivalencia axiomática desde la perspectiva de la teoría de la prueba y estudiar su relación con la equivalencia definida mediante teoría de modelos. En particular, sería necesario analizar las propiedades de corrección y completitud del sistema lógico utilizado para expresar propiedades de los programas. Si dichas propiedades se verifican para el lenguaje COMM, podría plantearse una caracterización unificada de la equivalencia axiomática que permita demostrar la coincidencia entre ambas perspectivas.

REFERENCIAS

- [1] H. Abelson, G. J. Sussman, and J. Sussman, *Structure and Interpretation of Computer Programs*, 2nd ed. MIT Press, 1996. [Online]. Available: <https://web.mit.edu/6.001/6.037/sicp.pdf>
- [2] L. Lamport, "An axiomatic semantics of concurrent programming languages," in *Logics and Models of Concurrent Systems*, ser. NATO ASI Series, K. R. Apt, Ed. Springer, 1985, pp. 77–122. [Online]. Available: <https://lamport.azurewebsites.net/pubs/semantics.pdf>
- [3] C. Tozzi. (2025) Difference between a statement vs. expression in programming. TechTarget. [Online]. Available: <https://www.techtarget.com/searchapparchitecture/tip/Difference-between-a-statement-vs-expression-in-programming>
- [4] S. K. Lahiri, A. Murawski, O. Strichman, and M. Ulbrich, "Program equivalence (dagstuhl seminar 18151)," *Dagstuhl Reports*, vol. 8, no. 4, pp. 1–19, 2018. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/DagRep.8.4.1>
- [5] Fiveable Content Team. (2025) Type equivalence. Fiveable. [Online]. Available: <https://fiveable.me/formal-logic-ii/key-terms/type-equivalence>
- [6] K. C. Louden and K. A. Lambert, *Programming Languages: Principles and Practices*, 3rd ed. Cengage Learning, 2020.
- [7] A. M. Pitts, "Operational semantics and program equivalence," in *Applied Semantics: International Summer School, APPSEM 2000, Caminha, Portugal, September 9–15, 2000, Advanced Lectures*, ser. Lecture Notes in Computer Science, G. Barthe, P. Dybjer, and J. Saraiva, Eds. Springer, 2002, vol. 2395, pp. 378–412.
- [8] B. C. Pierce, A. A. de Amorim, C. Casinghino, M. Gaboardi, M. Greenberg, C. Hrițcu, V. Sjöberg, A. Tolmach, and B. Yorgey, *Programming Language Foundations*, 7th ed., ser. Software Foundations, B. C. Pierce, Ed. Electronic textbook, 2026, vol. 2. [Online]. Available: <http://softwarefoundations.cis.upenn.edu>
- [9] M. Soto Romero. (2025) Semántica dinámica. Notas de curso. [Online]. Available: https://lambdasspace.github.io/LDP/notas/ldp_n06.pdf
- [10] H. R. Nielson and F. Nielson, *Semantics with Applications: A Formal Introduction*. Chichester: Wiley, 1992.
- [11] G. Hutton, "Programming language semantics: It's easy as 1,2,3," *Journal of Functional Programming*, vol. 33, p. e9, 2023. [Online]. Available: <https://doi.org/10.1017/S0956796823000072>
- [12] Cornell University, "Lecture 09: Hoare logic," <https://www.cs.cornell.edu/courses/cs4110/2014fa/lectures/lecture09.pdf>, 2014, cS 4110: Programming Languages and Logics.
- [13] J. Aldrich, "Notes 11: Hoare logic," <https://www.cs.cmu.edu/~aldrich/courses/17-355-18sp/notes/notes11-hoare-logic.pdf>, 2018, course notes for 17-355/17-665, Carnegie Mellon University.
- [14] M. Manzano, *Teoría de modelos*, ser. Alianza Universidad Textos. Madrid: Alianza Editorial, 1989. [Online]. Available: https://www.researchgate.net/profile/Maria-Manzano-Arjona/publication/277269984_Teoria_de_Modelos/links/5637c34708ae51ccb3cbea59/Teoria-de-Modelos.pdf
- [15] G. Winskel, *The Formal Semantics of Programming Languages: An Introduction*. Cambridge, MA: MIT Press, 1993.
- [16] A. S. Troelstra and H. Schwichtenberg, *Basic Proof Theory*, 2nd ed., ser. Cambridge Tracts in Theoretical Computer Science. Cambridge: Cambridge University Press, 2000, no. 43.
- [17] A. Bauer. The programming languages zoo. Programming Languages Zoo. [Online]. Available: <https://plzoo.andrej.com/language/comm.html>